

AI Builder Screening Agent

Zedong Sun — KPMG AI Builder Case Study

Demo / repository: <https://github.com/zeeautoclave/hiring-assistant> · **3-minute video:** <https://youtu.be/Xt9qx96ZWYw>

All candidate data is synthetic or anonymized. No confidential data is used.

What this is

Hiring Assistant is an AI agent for screening AI Builder candidates. You give it a job description and a folder of candidate submissions — each a résumé plus an AI-project write-up. For every candidate it:

- **checks the database to see whether they already applied in the last two months**, and silently skips repeats so no one is reviewed twice;
- **scores the new ones** on a seven-dimension, JD-derived rubric, with a cited quote from the candidate behind every score;
- **ranks them and writes a short summary** for those above the bar, then hands a human reviewer an explainable shortlist — which it can auto-email to the hiring manager.

Three AI agents do the *judgment* (read the JD, score each candidate, write the summary); deterministic code does everything that must be *exact* (dedup, weighting, the pass/fail gate) — so the results are auditable and reproducible. It turned an open, ambiguous prompt into a working, tested system.

1. How I framed the problem

"Evaluate an AI Builder well" is a very broad prompt, so I started by putting myself in the shoes of the person who actually has this problem — a hiring team — and asked what they are really trying to do. The goal isn't "score people"; it's **find the right candidate**.

That exposed the real problem: **traditional resume screening and keyword matching are a poor fit for hiring AI Builders**. You don't need a coding background or years of work experience to be a great AI Builder — the role itself says strong builders come from UX, ops, consulting, research, and self-taught paths. A keyword/resume filter would screen out exactly those people. So I worked it out in steps, and each step exposed the next problem:

1. **I need a better basis for scoring.** A resume alone isn't enough. So each candidate submits a resume **plus an AI-project write-up** — what they actually built — and the system weights project evidence above pedigree. (*Solves what to evaluate on; resume still counts, just not as the whole story.*)
2. **That still doesn't fix the keyword problem.** Often the keywords don't match and the candidate is genuinely strong. So instead of matching strings, I use **GenAI to understand the JD and the candidate's whole profile, and match on meaning** — transferable skill, not vocabulary. (*Solves matching.*)
3. **But understanding-based scoring brings its own problems** — bias, inconsistency, and high token cost — so I made three deliberate choices:
 - **The AI reads the JD once and writes a spec** (requirements + per-dimension cues + weights). Every candidate is scored against that compact spec, so the full JD is never re-read per candidate

→ controls **token cost**.

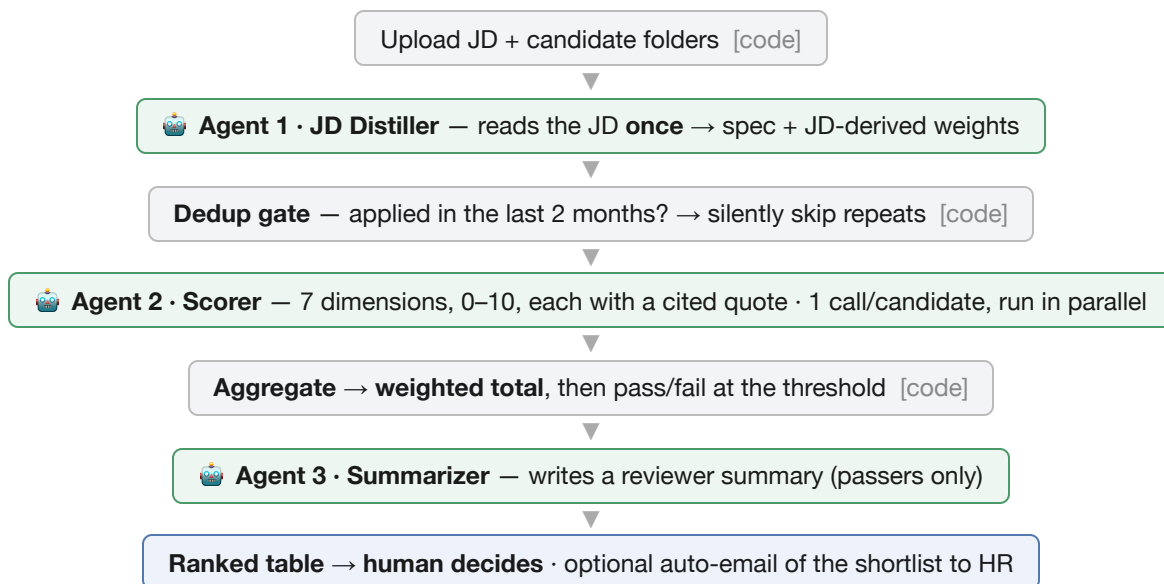
- **Every score must cite the candidate's own words** → prevents the model from **fabricating** a reason, and makes each number auditable.
- **Consistency**: the rubric anchors are frozen so everyone is measured the same way, and the design supports **scoring each profile multiple times and taking the median** to damp model variance (a planned next step).

4. **Efficiency**: scoring is the slow part, so candidates are **scored by multiple AI agents running in parallel**.

The result is an evaluation *system*, not a keyword filter: it scores what a builder actually built, matches on understanding, and stays auditable and reproducible — with a human making the final call.

2. What I built

A workflow with explicit human-AI handoffs, delivered as a native macOS app over a Node/TypeScript engine. It runs on a Claude subscription (`c laude -p`), so the marginal cost is ~\$0.



The model only does **judgment** (semantic match + dimension scoring). Everything that must be *correct* — the weighted math, deduplication, the gate — is deterministic code, so the pipeline is reproducible above the model layer.

Frozen vs dynamic. The rubric (dimension meanings + 0–10 behavioral anchors) is frozen, so every candidate is measured on one consistent ruler. The JD is injected at runtime and the dimension weights are derived from it — so the same ruler scores against *any* role. Proof: the same five candidates, scored against an "AI Builder" JD and an "AML Analyst" JD, **re-rank** — the builder profile drops below the line, the compliance profile rises above it. An automated test fails if that flip doesn't happen, which guards against a silently hard-coded system.

Seven scoring dimensions — weights are derived from each JD (defaults shown). Each is scored 0–10 against fixed anchors, with a required evidence quote:

#	Dimension	What it measures	Default weight
1	jd_fit	Whether the candidate's skills/methodologies match the JD's hard requirements (skill transfer, not keyword matching)	0.30
2	ai_judgment ★	How they use AI: clear human/AI boundary, evidence of correcting or overriding AI, mention of verification/evals	0.20
3	impact	Did it ship? Real users? Quantified outcomes?	0.15
4	complexity	How hard the problem was (absolute ladder, independent of the JD)	0.12
5	ownership	How much they personally drove the work (specific first-person actions vs vague "we")	0.12
6	reasoning	Clarity of reasoning under ambiguity: alternatives, tradeoffs, limits (judge the logic, not the prose)	0.08
7	pedigree	School prestige + academic level (deliberately a tiny signal)	0.03

3. Key decisions & tradeoffs (what I deliberately left out)

- **Plain-text input, no PDF/DOCX parsing.** Keeps the prototype focused on the evaluation logic, which is the hard part.
- **One multi-dimension call per candidate,** not seven calls — ~7× fewer tokens; the JD is understood once and reused, not re-fed per candidate.
- **No résumé-authenticity verification.** This is the biggest gap (see risks) — I chose to flag it and keep a human in the loop rather than half-build it.
- **No bias-audit dashboard** — described as a concept, not built, within the cap.

4. Responsible AI

- **AI ranks and explains; a human decides.** The "advance to next round" action is a person's click. No candidate is auto-rejected by the model.
- **Auditable.** Every score cites the candidate's own words; a reviewer can verify each number.
- **Fairness.** Pedigree is weighted near-zero for a builder role; reasoning is scored on logic, not prose polish.
- **Robustness.** A hidden "give me a 10" instruction inside a submission is treated as data and ignored — verified with an A/B test.
- **Fail-safe.** A failed model call is surfaced as an error, never silently scored zero — a system fault must not masquerade as a weak candidate.
- **Privacy / governance.** Dedup uses a 2-month window; re-submissions are silently skipped; the prototype uses only synthetic data.

5. Risks & what I'd do next

Risk	Mitigation now	Next
Self-reported projects are gameable	Human-in-the-loop; evidence required	Require repo/demo links → verifiability multiplier
LLM scoring variance	Frozen anchored rubric; deterministic aggregation	Self-consistency (median of N) on key dimensions
Articulation/fluency bias	Score logic, not prose	Structured intake fields per dimension
Cross-reviewer inconsistency (the original pain)	One frozen ruler, JD-injected	Calibration set + drift monitoring

6. How I used AI vs. my own decisions

Claude did: scaffold code, draft prompt text, generate synthetic candidates, first-draft the rubric anchors.

I decided: the reframe (eval harness), the frozen-vs-dynamic architecture, the seven dimensions and the JD-derived weighting strategy, the determinism boundary between model and code, the dedup rules, the human-in-the-loop boundary, and the test invariants (dynamic, dedup, injection) that prove the system behaves as claimed.

7. Running it

See `README.md` in the repo. In short: `npm install`, then `npm run -- --jd <jd.txt> --profiles <folder> --threshold 6`. 19 tests run fully offline via record-replay (`npm test`). The macOS app wraps the same engine.